

Sistema de Recuperación de Información Multimodal con RAG

Asignatura: ICCD753 Recuperación de Información

Integrantes: Alexander Reyes y Francisco Hernandez

1. Descripción del Corpus Utilizado

El sistema construye su corpus multimodal uniendo dos datasets públicos de Hugging Face, ya que ninguno por sí solo aporta texto e imagen para el mismo producto:

- ``crossingminds/shopping-queries-image-dataset`` (SQID), config ``product_image_urls``: aporta únicamente el mapeo ``product_id -> image_url`` (fotografías reales de producto de Amazon).

- ``tasksource/esci`` (split ``test``): es la versión en Hugging Face del **Amazon Shopping Queries Dataset** (ESCI). Aporta la consulta (``query``), el ``product_id``, el título del producto (``product_title``) y el juicio de relevancia (``esci_label``).

``backend/corpus.py`` transmite (streaming) el dataset ESCI filtrando por ``product_locale == "us"`` y ``small_version == 1`` (el subconjunto curado usado en el paper original de ESCI para benchmarking), y por cada fila conserva el producto solo si existe una imagen real asociada en el mapeo de SQID - así se garantiza que el corpus indexado sea genuinamente multimodal (texto + imagen), en vez de mostrar imágenes que nunca participan en la recuperación. El proceso se detiene al reunir un número objetivo de consultas distintas (``NUM_QUERIES = 25`` por defecto, configurable en ``backend/corpus.py``).

- Estructura del documento indexado:

- ``product_id``: identificador alfanumérico del producto (ASIN de Amazon).

- ``title``: título comercial del producto (``product_title`` de ESCI), usado para generar el embedding de texto con la torre textual de CLIP.

- ``image_url``: URL directa a la fotografía del producto (de SQID). No es solo un campo de metadata para mostrar en la interfaz: en la etapa de indexación se descarga esa imagen y se genera su embedding de imagen con la torre visual de CLIP, que se fusiona con el embedding de texto (detalle completo en la Sección 2). Esto es lo que hace la recuperación genuinamente multimodal, y no solo texto con fotos de adorno.

- Juicios de relevancia (``qrels.json``): para cada consulta se listan los productos evaluados por anotadores humanos con su etiqueta ESCI, mapeada a un puntaje graduado para NDCG:

- Exact (concordancia exacta): 3 puntos.

- Substitute (sustituto directo): 2 puntos.

- Complement (complementario): 1 punto.

- Irrelevant (irrelevante): 0 puntos.

- Se descartan del corpus las consultas que, tras el filtro de imágenes, quedan sin ningún producto relevante (``score > 0``), ya que no aportarían señal útil a Precision/Recall/NDCG.

- Modo de respaldo: si no hay conexión a internet o la descarga falla, ``backend/corpus.py`` recae automáticamente en un corpus mock de 10 productos (``generate_mock_corpus()``), pensado únicamente para poder desarrollar/depurar sin conexión - el corpus de la entrega final es el real descrito arriba.

Tamaño real obtenido: al ejecutar el pipeline completo, el proceso descrito reunió 348 productos y 23 consultas distintas con al menos un producto relevante marcado (de las 25 consultas objetivo, 2 quedaron sin relevancia positiva tras el filtro de imágenes y se descartaron). Como las consultas se toman en el orden en que aparecen en el stream de ESCI (sin curaduría por tema), el corpus final resultó temáticamente diverso - desde ``"110cfm bathroom exhaust fan without light"`` (31 productos relevantes) y ``"fences"`` (3 relevantes) hasta ``"manilla envelopes 10x13"`` (14 relevantes) o ``"4 wheel go kart"`` (17 relevantes) - lo cual hace la evaluación más realista que un catálogo de nicho armado a mano.

2. Arquitectura General del Sistema

El sistema implementa una arquitectura modular desacoplada mediante un cliente web (Frontend) y un servidor de procesamiento semántico (Backend) comunicados a través de una API REST:

```
graph TD
    subgraph Frontend [Next.js Web Client]
        UI[Chat Interface: Josefin Sans]
        Evidencias[Visor de Evidencias Multimodales]
        Feedback[Botones de Relevance Feedback]
    end

    subgraph Backend [FastAPI Python Server]
        API[FastAPI Endpoints]
        QE[Query Expansion: Gemini]
        CLIP_TXT[CLIP Texto: clip-ViT-B-32]
        CLIP_IMG[CLIP Imagen: clip-ViT-B-32]
        FAISS[Vector Store: FAISS Index]
        RERANK[Re-ranking: Cross-Encoder]
        LLM[RAG Generator: Gemini Flash]
        FeedbackDB[Base de Datos de Feedback]
        Eval[Módulo de Evaluación]
    end

    UI -->|1. Petición /api/chat| API
    Feedback -->|Petición /api/feedback| API
    API --> QE
    QE --> CLIP_TXT
    CLIP_IMG -->|Embedding fusionado en indexación| FAISS
    CLIP_TXT -->|Embedding de consulta| FAISS
    FAISS -->|Top-8 Candidatos| RERANK
    RERANK -->|Top-4 Reordenados| LLM
    FeedbackDB -->|Ajuste de Scores| RERANK
    LLM -->|2. Respuesta + Evidencias + Similitudes| UI
```

- Frontend (Next.js): construido sobre React, TypeScript y Tailwind CSS. Implementa un diseño monocromático y tipografía Josefin Sans. Integra un panel colapsable (<details>) bajo cada respuesta para inspeccionar las evidencias (título, miniatura de imagen, ID de producto y similitud coseno), con botones individuales de "me gusta"/"no me gusta" por evidencia.
- Backend (FastAPI): orquesta el pipeline de recuperación vectorial y generación. Utiliza FAISS como motor de indexación de vectores en memoria (IndexFlatIP) y la API de Google Gemini como modelo generativo, con una lista de modelos de respaldo ante error de cuota (ver Sección 3).
- Embeddings multimodales reales (backend/embeddings.py, backend/vector_db.py): durante la indexación, cada producto se representa con la fusión de su embedding de texto (título) y su embedding de imagen (ambos generados por las dos torres del mismo modelo clip-ViT-B-32, que comparten espacio vectorial): fusionado = normalizar(emb_texto + emb_imagen). Si la imagen no se puede descargar (URL caída, timeout), el producto se indexa solo con su embedding de texto - nunca se descarta un producto por eso. Esto hace que la similitud coseno usada en la búsqueda compare la consulta contra una representación que sí incorpora la imagen del producto, cumpliendo la "recuperación multimodal" pedida por el enunciado y no solo mostrando la imagen como adorno visual.

3. Pipeline de Recuperación y Generación (RAG)

El flujo de procesamiento de una consulta (backend/rag.py:generate_rag_response) se ejecuta en los siguientes pasos:

1. Expansión de consulta (Query Expansion - excelencia): la consulta cruda del usuario se reescribe con Gemini para incluir sinónimos y términos relacionados de e-commerce, mejorando la recuperación ante vocabulario desalineado entre el usuario y el catálogo.

2. Embedding de la consulta (CLIP, torre de texto): la consulta expandida se codifica con el modelo ``clip-ViT-B-32`` (``sentence-transformers``), normalizando el vector resultante (``normalize_embeddings=True``). Del lado del corpus, cada producto ya fue indexado previamente con un embedding fusionado de texto + imagen (ver Sección 2) generado con las dos torres del mismo modelo CLIP - texto y visión comparten el mismo espacio vectorial, por lo que la comparación coseno entre ambos es válida.
3. Búsqueda vectorial (FAISS): se calcula la similitud coseno (producto interno sobre vectores L2-normalizados, ``IndexFlatIP``) entre el embedding de la consulta y el embedding multimodal de cada producto del corpus, recuperando los 8 candidatos más cercanos.
4. Ajuste por Relevance Feedback (excelencia): si existe retroalimentación previa del usuario ("me gusta"/"no me gusta") para ese par consulta-producto, se ajusta linealmente (± 0.1) el score de similitud antes del re-ranking.
5. Re-ranking (Cross-Encoder - excelencia): los candidatos se evalúan por pares ``(consulta, título_producto)`` con ``cross-encoder/ms-marco-MiniLM-L-6-v2``, que captura relevancia textual profunda no disponible en el encoder de una sola torre de CLIP. Se seleccionan los 4 mejores productos reordenados.
6. Generación aumentada (RAG): se construye un prompt que inyecta el historial de conversación (memoria - excelencia) y el contexto de los 4 productos (título, ID, score de recuperación). El LLM genera la respuesta final basándose exclusivamente en ese contexto.

Modelo de lenguaje y resiliencia: ``call_gemini_with_fallback`` intenta, en orden, la lista ``GEMINI_MODELS`` definida en ``backend/rag.py`` (actualmente ``gemini-3.1-flash-lite -> gemini-2.0-flash-lite -> gemini-2.0-flash -> gemini-1.5-flash -> gemini-1.5-flash-8b``), con reintentos y backoff ante error 429 (cuota excedida) y salto automático al siguiente modelo ante error 404. Si todos los modelos fallan, el sistema responde con un mensaje de degradación controlada mostrando igualmente los productos recuperados, sin romper la experiencia del usuario.

4. Resultados Experimentales y Análisis de Métricas

La evaluación (``backend/evaluation.py``) contrasta la recuperación de FAISS contra los juicios de relevancia de ``qrels.json``, reportando `Precision@k`, `Recall@k` y `NDCG@k` (graduado, con ganancia $2^{\{rel\}-1}$) para k in {1, 3, 5}. Se calculan dos variantes para poder cuantificar el aporte de la funcionalidad de excelencia de Re-ranking:

- Baseline: ranking crudo de CLIP + FAISS (``run_evaluation(apply_reranking=False)``).
- Con Re-ranking: se recupera un pool más amplio con FAISS y se reordena con el Cross-Encoder antes de recortar a cada k (``run_evaluation(apply_reranking=True)``), reutilizando el mismo modelo que usa el pipeline de producción.

Evaluado sobre el corpus real (348 productos, 23 consultas; Sección 1):

Tabla A - Baseline (solo CLIP + FAISS)

Umbral (k)	Precision@k	Recall@k	NDCG@k
k = 1	0.2609	0.0210	0.2112
k = 3	0.2899	0.0811	0.2516
k = 5	0.2609	0.1515	0.2561

Tabla B - Con Re-ranking (Cross-Encoder)

Umbral (k)	Precision@k	Recall@k	NDCG@k
k = 1	0.4783	0.0906	0.4161
k = 3	0.3623	0.1442	0.3624
k = 5	0.3391	0.1784	0.3564

Análisis y Discusión de Resultados

- El Re-ranking mejora Precision@k y NDCG@k en los tres umbrales, de forma más marcada en k=1: Precision sube de 0.2609 a 0.4783 (+83% relativo) y NDCG casi se duplica (0.2112 -> 0.4161). Esto confirma cuantitativamente que la comparación cruzada del Cross-Encoder (que evalúa el par consulta-título completo, no vectores independientes de una sola torre de CLIP) ordena las coincidencias `Exact` por encima de sustitutos y complementos con mucha más precisión que el ranking crudo de FAISS.
- Recall@k es bajo en términos absolutos (0.02-0.18) pero crece monótonamente con k en ambas tablas, como se espera. La razón no es una falla del sistema: al provenir de un benchmark real (ESCI), varias consultas tienen decenas de productos marcados como relevantes (hasta 31 en algunos casos) - el techo matemático de Recall@5 para una consulta con 31 relevantes es $5/31 \approx 0.16$, así que un Recall@5 de 0.15-0.18 está cerca de ese techo, no lejos de un ideal de 1.0. Esto es más representativo de una evaluación de RI real que un catálogo de juguete con 3-4 relevantes por consulta (donde Recall@3=1.0 se alcanza trivialmente).
- El Re-ranking también mejora ligeramente el Recall@k (0.1515 -> 0.1784 en k=5): al re-ranear se parte de un pool de candidatos más amplio (2k) antes de recortar a los mejores k, por lo que además de reordenar tiene una oportunidad extra de traer a la ventana final productos relevantes que el ranking crudo de FAISS había dejado justo fuera del top-k.
- Precision@k no decrece monótonamente con k en la Tabla A (0.2609 to 0.2899 to 0.2609), consistente con consultas donde el segundo o tercer resultado de FAISS es tan relevante como el primero; en la Tabla B sí decrece de forma más ordenada (0.4783 -> 0.3623 -> 0.3391), reflejo de que el re-ranking concentra mejor la relevancia en las primeras posiciones - el refinamiento de grano fino que se espera de esta funcionalidad de excelencia.

5. Descripción de las Funcionalidades de Excelencia Implementadas

Se incorporaron 4 funcionalidades avanzadas sobre el sistema base:

1. Re-ranking (+15 puntos): `cross-encoder/ms-marco-MiniLM-L-6-v2` reordena los candidatos de FAISS evaluando pares `(consulta, título)` directamente, corrigiendo casos donde la similitud de embeddings de una sola torre no captura bien la relevancia textual fina. Implementado en `backend/rag.py` (pipeline de producción) y reutilizado en `backend/evaluation.py` para poder medir su impacto cuantitativamente (Sección 4).
2. Query Expansion (+15 puntos): Gemini reescribe la consulta del usuario agregando sinónimos y términos de e-commerce relacionados antes de generar el embedding, mitigando el desajuste de vocabulario entre la consulta y los títulos del catálogo (`expand\_query\_with\_llm` en `backend/rag.py`).
3. Relevance Feedback (+15 puntos): los botones de "me gusta"/"no me gusta" de cada evidencia en la UI notifican a `POST /api/feedback`, que persiste el feedback en `backend/data/feedback.json` (clave `consulta||product\_id`). `apply\_relevance\_feedback` ajusta ± 0.1 el score de similitud de ese par en búsquedas futuras con la misma consulta.
4. Memoria Conversacional (+15 puntos): el frontend envía el historial de la sesión en cada petición a `/api/chat`; el backend inyecta los últimos 6 mensajes en el prompt de generación para mantener el hilo del diálogo y resolver referencias contextuales.